

Python OOP — To'liq Kurs Qo'llanmasi

Obyektga yo'naltirilgan dasturlash: klassdan vorislikgacha — diagrammalar va sinovdan o'tgan kodlar bilan

Mavzu	Python'da OOP: klass, obyekt, inkapsulyatsiya, vorislik, polimorfizm, abstraksiya
Hajmi	4-6 dars (~10-12 akademik soat)
Oldingi bilim	Python asoslari: o'zgaruvchi, shart, sikl, funksiya, ro'yxat va lug'at
Kurs maqsadi	O'quvchi o'z klasslarini loyihalab, vorislik va polimorfizmdan real loyihada foydalana olishi
Muhim izoh	Qo'llanmadagi barcha kodlar Python 3 da ishga tushirilgan — natijalar haqiqiy terminal chiqishi

1. Kurs rejasi

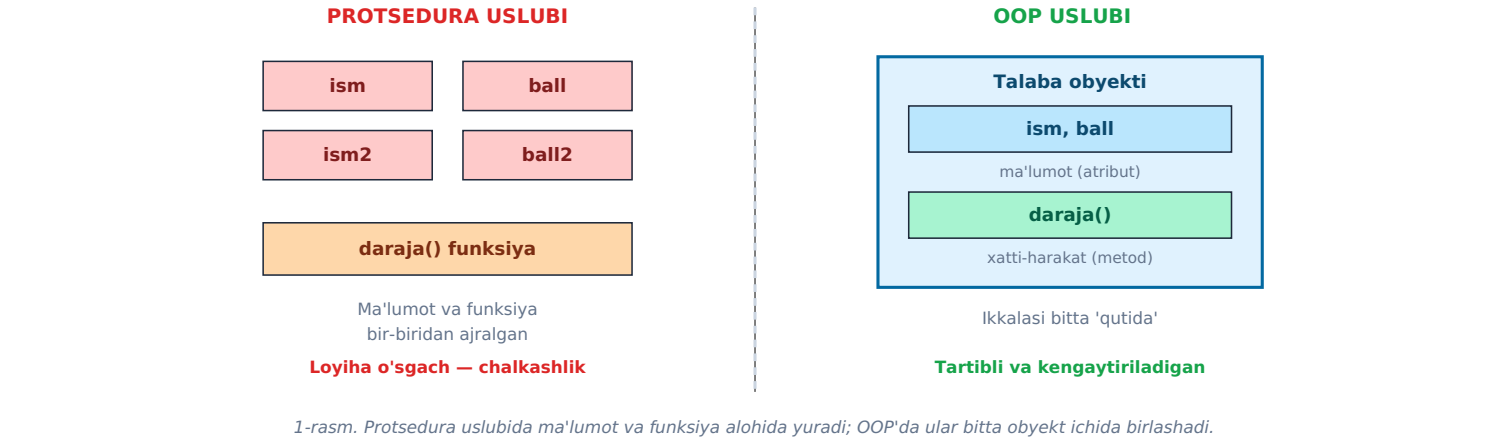
Dars	Mavzu	Qamrab olinadigan tushunchalar
1-dars	OOP nima va nega kerak?	Protsedura uslubi muammosi, klass va obyekt tushunchasi, __init__, self, atributlar va metodlar
2-dars	Inkapsulyatsiya	public / _protected / __private, getter-setter, @property dekoratori
3-dars	Vorislik (meros olish)	Ota va voris klasslar, super(), vorislik turlari, MRO va olmos muammosi
4-dars	Polimorfizm	Metodni qayta yozish (overriding), duck typing, bitta interfeys - ko'p ko'rinish
5-dars	Abstraksiya	ABC moduli, @abstractmethod, 'shartnoma' tushunchasi
6-dars	Qo'shimcha imkoniyatlar	Magic (dunder) metodlar, @staticmethod, @classmethod, yakuniy loyiha

2. OOP nima va nega kerak?

MUAMMODAN BOSHLAYMIZ

Hozirgacha siz **protsedura uslubida** yozgansiz: o'zgaruvchilar alohida, funksiyalar alohida. Kichik dasturda bu yaxshi ishlaydi. Lekin loyiha o'sgach muammo boshlanadi: 100 ta talaba bo'lsa, 200 ta o'zgaruvchi yozamizmi? Talabaga yangi maydon (telefon raqami) qo'shilsa, necha joyni tuzatamiz?

OOP (Object-Oriented Programming — obyektga yo'naltirilgan dasturlash) shu muammoni hal qiladi: u **ma'lumot** (atribut) va **u bilan bajariladigan amallar** (metod) ni bitta "qutiga" — obyektga joylaydi.



Protsedura uslubi

```
protsedura_uslubi.py

# Protsedura uslubi — ma'lumot va funksiya alohida
talaba1_ism = "Aziz"
talaba1_ball = 85

talaba2_ism = "Malika"
talaba2_ball = 92

def daraja(ball):
    if ball >= 90:
        return "A'lo"
    elif ball >= 80:
        return "Yaxshi"
    return "Qoniqarli"
```

```
print(talaba1_ism, "-", daraja(talaba1_ball))
print(talaba2_ism, "-", daraja(talaba2_ball))
```

Natija (terminalda ishga tushirilgan)

```
Aziz - Yaxshi
Malika - A'lo
```

Xuddi shu vazifa — OOP uslubida

oop_uslub.py

```
class Talaba:
    def __init__(self, ism, ball):
        self.ism = ism
        self.ball = ball

    def daraja(self):
        if self.ball >= 90:
            return "A'lo"
        elif self.ball >= 80:
            return "Yaxshi"
        return "Qoniqarli"

t1 = Talaba("Aziz", 85)
t2 = Talaba("Malika", 92)

print(t1.ism, "-", t1.daraja())
print(t2.ism, "-", t2.daraja())
print(type(t1))
```

Natija (terminalda ishga tushirilgan)

```
Aziz - Yaxshi
Malika - A'lo
<class '__main__.Talaba'>
```

Farqni sezingizmi? Endi yangi talaba qo'shish uchun bitta qator yetarli: `Talaba("Jasur", 78)` . Yangi maydon qo'shish kerak bo'lsa — faqat klassni o'zgartiramiz, qolgan hamma joyga avtomatik tarqaladi.

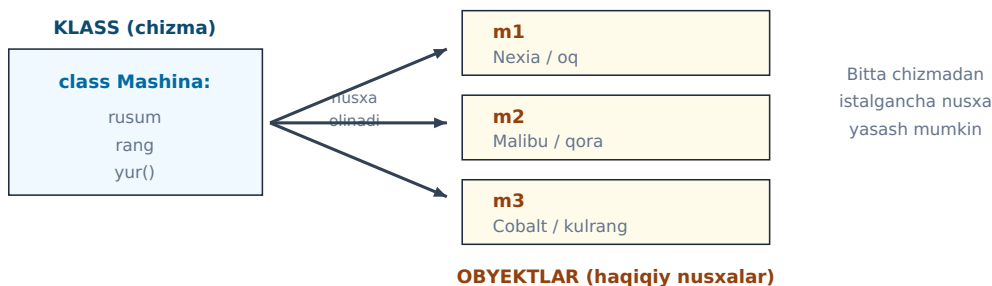
3. Klass va obyekt

ASOSIY TUSHUNCHA

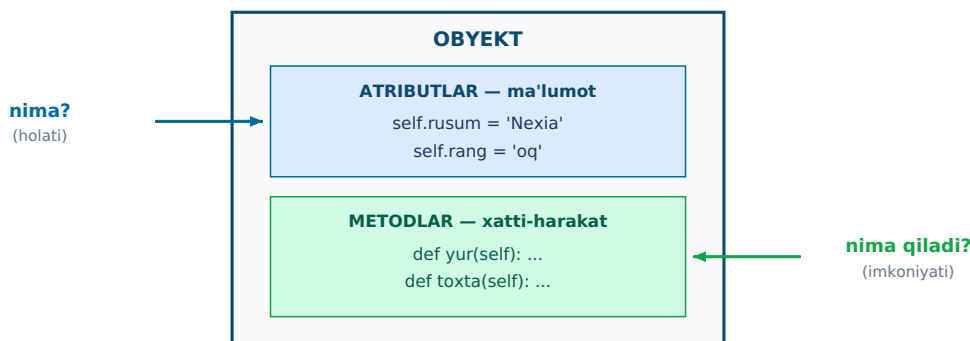
Klass (class) — bu *chizma*. U hech narsa emas, faqat ta'rif: "mashina degani nima, uning qanday xususiyatlari va imkoniyatlari bo'ladi".

Obyekt (object) — shu chizma asosida yasalgan *haqiqiy nusxa*. Bitta chizmadan minglab nusxa yasash mumkin, va har biri o'z qiymatlariga ega bo'ladi.

Uyni misol qilsak: **loyiha chizmasi** — klass, **shu chizma bo'yicha qurilgan har bir uy** — obyekt.



2-rasm. Klass — bu chizma (blueprint). Obyekt — shu chizma asosida yasalgan haqiqiy nusxa.



3-rasm. Har bir obyekt ikki narsadan iborat: atributlar (u nima) va metodlar (u nima qila oladi).

3.1. `__init__` va `self` nima?

NIMA UCHUN KERAK?

`__init__` — **konstruktor** deb ataladi. Obyekt yaratilgan payt Python uni **avtomatik** chaqiradi. Uning vazifasi — yangi obyektning boshlang'ich qiymatlarini o'rnatish.

`self` — **obyektning o'ziga ishora**. Har bir metodning birinchi parametri sifatida yoziladi. U orqali Python "qaysi obyekt haqida gapiryapmiz"ni biladi. `t1.daraja()` deb yozganda Python buni ichkarida `Talaba.daraja(t1)` ga aylantiradi — ya'ni `self` o'rniga `t1` keladi.

Eng ko'p uchraydigan savol: "self ni nega har safar yozamiz?" — chunki bitta klassdan 1000 ta obyekt yaratilishi mumkin, va metod qaysi biri bilan ishlayotganini bilishi shart. `self` — aynan shu manzil.

3.2. Klass atributi va obyekt atributi

FARQI NIMADA?

Obyekt atributi (`self.rang`) — har bir obyekt uchun *alohida*. Birini o'zgartirsak, boshqalariga ta'sir qilmaydi.

Klass atributi (`gildiraklar_soni`) — barcha obyektlar uchun *umumiy*. Klass darajasida o'zgartirilsa, hamma obyektga tarqaladi. Odatda o'zgarmas qiymatlar (konstantalar) uchun ishlatiladi.

atributlar.py

```
class Mashina:
    # Klass atributi — barcha obyektlar uchun umumiy
    gildiraklar_soni = 4

    def __init__(self, rusum, rang):
        # Obyekt atributlari — har bir obyekt uchun alohida
```

```

        self.rusum = rusum
        self.rang = rang

m1 = Mashina("Nexia", "oq")
m2 = Mashina("Malibu", "qora")

print(m1.rusum, m1.rang, m1.gildiraklar_soni)
print(m2.rusum, m2.rang, m2.gildiraklar_soni)

# Klass atributini o'zgartirsak – hammasiga ta'sir qiladi
Mashina.gildiraklar_soni = 6
print("O'zgartirgandan keyin:", m1.gildiraklar_soni, m2.gildiraklar_soni)

# Obyekt atributi esa faqat o'ziga tegishli
m1.rang = "qizil"
print(m1.rang, "|", m2.rang)

```

Natija (terminalda ishga tushirilgan)

```

Nexia oq 4
Malibu qora 4
O'zgartirgandan keyin: 6 6
qizil | qora

```

3.3. Metodlar — obyektning xatti-harakati

NIMA UCHUN KERAK?

Metod — klass ichida yozilgan funksiya. Oddiy funksiyadan farqi: u obyektning o'z ma'lumotlari bilan ishlaydi (`self` orqali). Shuning uchun metod obyektning "nima qila olishi"ni bildiradi.

metodlar.py

```

class Hisob:
    def __init__(self, egasi, balans=0):
        self.egasi = egasi
        self.balans = balans

    def kirim(self, summa):
        self.balans += summa
        return self.balans

    def chiqim(self, summa):
        if summa > self.balans:
            return "Mablag' yetarli emas!"
        self.balans -= summa
        return self.balans

h = Hisob("Salom", 100000)
print("Boshlang'ich:", h.balans)
print("Kirimdan keyin:", h.kirim(50000))
print("Chiqimdan keyin:", h.chiqim(30000))
print("Katta chiqim:", h.chiqim(999999))

```

Natija (terminalda ishga tushirilgan)

```

Boshlang'ich: 100000
Kirimdan keyin: 150000
Chiqimdan keyin: 120000
Katta chiqim: Mablag' yetarli emas!

```

4. OOP'ning to'rt tayanch ustuni

OOP'NING 4 TAYANCH USTUNI



4-rasm. OOP'ning to'rt tayanch ustuni — qo'llanmaning qolgan qismi shu to'rttasini ochib beradi.

QISQACHA

- **Inkapsulyatsiya** — muhim ma'lumotni tashqi aralashuvdan himoyalash.
- **Vorislik** — mavjud klassdan meros olib, kodni qayta yozmaslik.
- **Polimorfizm** — bir xil metod nomi turli obyektlarda turlicha ishlashi.
- **Abstraksiya** — murakkab ichki mantiqni yashirib, faqat kerakli qismini ko'rsatish.

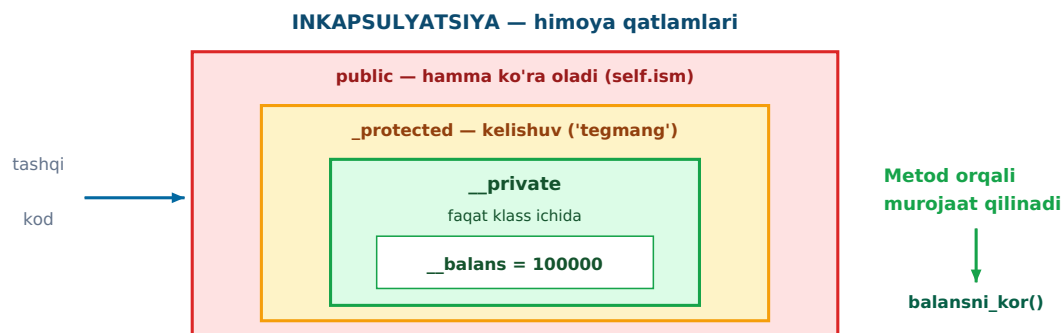
5. Inkapsulyatsiya (ma'lumotni himoyalash)

NIMA UCHUN KERAK?

Tasavvur qiling: bank hisobi obyektining balansiga istalgan joydan to'g'ridan-to'g'ri murojaat qilib `hisob.balans = 999999999` deb yozish mumkin bo'lsa — bu falokat. Inkapsulyatsiya shuni oldini oladi: muhim ma'lumot yashiriladi, unga faqat **nazorat qiluvchi metodlar** orqali murojaat qilinadi.

Python'da uch daraja mavjud:

- `self.ism` — **public**: hamma ko'radi, hamma o'zgartiradi.
- `self._kod` — **protected**: bu shunchaki *kelishuv* ("tegman" degan ogohlantirish). Python texnik jihatdan to'smaydi.
- `self.__balans` — **private**: Python nomni ichkarida o'zgartirib qo'yadi (*name mangling*), shuning uchun tashqaridan oddiy yo'l bilan murojaat qilib bo'lmaydi.



5-rasm. Muhim ma'lumot ichkarida himoyalanaadi; tashqaridan unga faqat metodlar orqali murojaat qilinadi.

inkapsulyatsiya.py

```
class BankHisobi:
    def __init__(self, egasi, balans):
        self.egasi = egasi          # public
        self._bank_kodi = "UZ001"  # protected (kelishuv)
        self.__balans = balans      # private (nom o'zgartiriladi)

    def balansni_kor(self):
        return self.__balans

    def kirim(self, summa):
        if summa <= 0:
            return "Xato: summa musbat bo'lishi kerak"
        self.__balans += summa
        return self.__balans

h = BankHisobi("Salom", 100000)
print("Public:", h.egasi)
```

```

print("Protected:", h._bank_kodi)
print("Metod orqali balans:", h.balansni_kor())

# Private atributga to'g'ridan-to'g'ri murojaat
try:
    print(h._balans)
except AttributeError as e:
    print("XATO:", e)

# Python aslida nomni o'zgartirib qo'yadi
print("Yashirin nom orqali:", h._BankHisobi__balans)

```

Natija (terminalda ishga tushirilgan)

```

Public: Salom
Protected: UZ001
Metod orqali balans: 100000
XATO: 'BankHisobi' object has no attribute '__balans'
Yashirin nom orqali: 100000

```

Muhim haqiqat: natijadagi oxirgi qatorga qarang — `h._BankHisobi__balans` orqali private atributga baribir yetib borish mumkin! Ya'ni Python'dagi private — *qattiq qulf emas, balki ogohlantiruvchi belgi*. Bu Python falsafasi: "biz hammamiz kattalarmiz" (we are all consenting adults).

5.1. @property — eng chiroyli yechim

NIMA UCHUN KERAK?

Getter va setter metodlarini yozish mumkin, lekin shunda kod `h.selsiy_ol()` kabi noqulay ko'rinadi. **@property** dekoratori metodni *oddiy atribut kabi* ishlatish imkonini beradi: `h.selsiy` deb yozasiz, lekin orqa fonda metod ishlaydi va tekshiruv o'tkazadi.

Bu — inkapsulyatsiyaning Python'dagi eng zamonaviy va tavsia etiladigan usuli.

property_dekoratori.py

```

class Harorat:
    def __init__(self, selsiy=0):
        self._selsiy = selsiy

    @property
    def selsiy(self):
        return self._selsiy

    @selsiy.setter
    def selsiy(self, qiymat):
        if qiymat < -273.15:
            raise ValueError("Absolyut noldan past bo'lishi mumkin emas!")
        self._selsiy = qiymat

    @property
    def farengeyt(self):
        return self._selsiy * 9 / 5 + 32

h = Harorat(25)
print("Selsiy:", h.selsiy)
print("Farengeyt:", h.farengeyt)

h.selsiy = 100
print("Yangi selsiy:", h.selsiy, "| Farengeyt:", h.farengeyt)

try:
    h.selsiy = -300
except ValueError as e:
    print("XATO:", e)

```

Natija (terminalda ishga tushirilgan)

```

Selsiy: 25
Farengeyt: 77.0
Yangi selsiy: 100 | Farengeyt: 212.0
XATO: Absolyut noldan past bo'lishi mumkin emas!

```

6. Vorislik — meros olish

NIMA UCHUN KERAK?

Faraz qiling, `It`, `Mushuk`, `Ot` klasslarini yozyapsiz. Har birida `nafas_olish()`, `ovqatlanish()`, `uxlash()` metodlari bir xil bo'ladi — uch marta bir xil kod yozasizmi?

Vorislik (inheritance) shu takrorlanishni yo'q qiladi: umumiy narsalarni bitta **ota klass**ga (`Hayvon`) yozib qo'yasiz, qolganlari undan **meros oladi**. Har bir voris faqat o'ziga xos qismini yozadi.

Bu OOP'ning eng kuchli va eng ko'p ishlatiladigan imkoniyati.

vorislik_oddiy.py

```
class Hayvon:
    def __init__(self, ism):
        self.ism = ism

    def nafas_olish(self):
        return f"{self.ism} nafas olmoqda"

    def ovoz(self):
        return "..."
```

```
class It(Hayvon):          # It <- Hayvon dan meros oladi
    def ovoz(self):
        return "Vov-vov!"

k = It("Bobik")
print(k.nafas_olish())    # ota klassdan meros
print(k.ovoiz())          # o'zi qayta yozgan
print(isinstance(k, Hayvon))
print(issubclass(It, Hayvon))
```

Natija (terminalda ishga tushirilgan)

```
Bobik nafas olmoqda
Vov-vov!
True
True
```

KODNI TUSHUNISH

- `class It(Hayvon)` — qavs ichidagi nom ota klassni bildiradi.
- `nafas_olish()` metodi `It` da yozilmagan, lekin ishladi — chunki u **meros qilib olindi**.
- `ovoiz()` metodi esa qayta yozilgan — bu **method overriding** (metodni qayta belgilash) deyiladi.
- `isinstance(k, Hayvon) → True` : it ayni paytda hayvon hamdir.

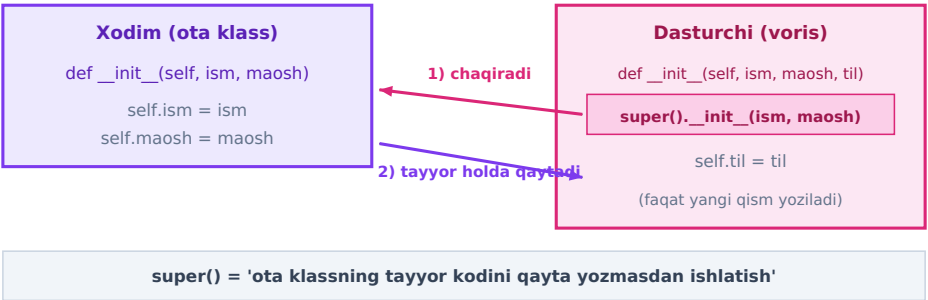
6.1. super() — ota klassga murojaat

NIMA UCHUN KERAK?

Voris klassda `__init__` yozsak, ota klassning `__init__` i **ishlamay qoladi** — ya'ni ota belgilagan atributlar o'rnatilmaydi. Natijada `AttributeError` chiqadi.

super() aynan shuni hal qiladi: ota klassning metodini chaqiradi, shundan keyin biz faqat *yangi qo'shilgan qismni* yozamiz. Kod takrorlanmaydi.

super() qanday ishlaydi?



7-rasm. `super()` ota klassning metodini chaqiradi — kodni ikki marta yozishdan saqlaydi.

super_metodi.py

```
class Xodim:
    def __init__(self, ism, maosh):
        self.ism = ism
        self.maosh = maosh

    def malumot(self):
        return f"{self.ism} | maosh: {self.maosh}"

class Dasturchi(Xodim):
    def __init__(self, ism, maosh, til):
        super().__init__(ism, maosh) # ota klass __init__ ini chaqiramiz
        self.til = til

    def malumot(self):
        asos = super().malumot() # ota metodini ishlatib, ustiga qo'shamiz
        return f"{asos} | til: {self.til}"

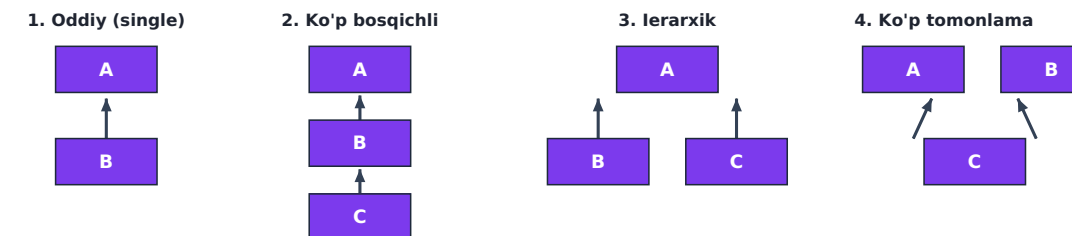
d = Dasturchi("Salom", 5000000, "Python")
print(d.malumot())
print(d.ism, d.maosh, d.til)
```

Natija (terminalda ishga tushirilgan)

```
Salom | maosh: 5000000 | til: Python
Salom 5000000 Python
```

6.2. Vorislik turlari

VORISLIK TURLARI



Strelka yo'nalishi: voris → ota klass (ya'ni 'kimdan meros oldi')

6-rasm. Vorislikning to'rt asosiy turi. Python to'rttasini ham qo'llab-quvvatlaydi.

kop_bosqichli_vorislik.py

```
class Transport:
    def harakat(self):
        return "Harakatlanmoqda"

class Mashina(Transport):
    def gildirak(self):
        return "4 ta gildirak"

class ElektroMashina(Mashina):
    def quvvat(self):
        return "Batareyadan quvvat oladi"

e = ElektroMashina()
print(e.harakat()) # Transport dan
print(e.gildirak()) # Mashina dan
print(e.quvvat()) # o'zidan

print([c.__name__ for c in ElektroMashina.__mro__])
```

Natija (terminalda ishga tushirilgan)

```
Harakatlanmoqda
4 ta gildirak
Batareyadan quvvat oladi
['ElektroMashina', 'Mashina', 'Transport', 'object']
```

Natijaning oxirgi qatoriga qarang: `__mro__` — bu Python metodni qaysi tartibda qidirishini ko'rsatadi. `ElektroMashina → Mashina → Transport → object`. Har bir Python klassi oxir-oqibat `object` dan meros oladi.

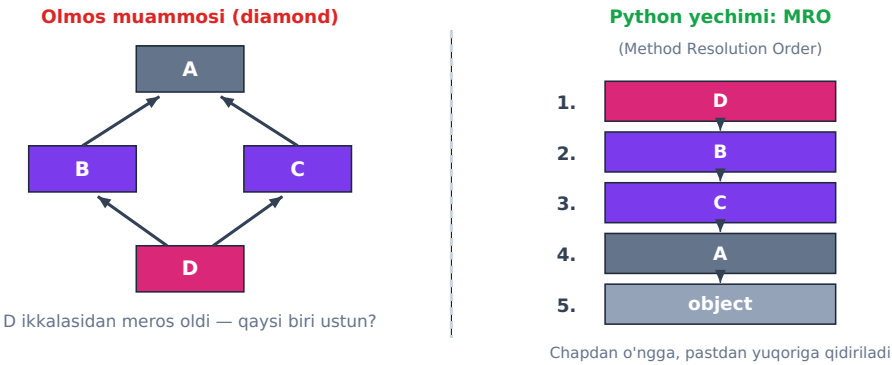
6.3. MRO va olmos muammosi

MUAMMO NIMADA?

Agar klass **ikkita** ota klassdan meros olsa va ikkalasida ham bir xil nomli metod bo'lsa — qaysi biri ishlaydi? Bu klassik muammo "**olmos**"

muammosi" (diamond problem) deb ataladi, chunki klasslar sxemasi olmos shaklini oladi.

Python buni **MRO (Method Resolution Order — metodni aniqlash tartibi)** orqali hal qiladi: aniq, oldindan bashorat qilinadigan tartib tuziladi va metod shu tartib bo'yicha qidiriladi. Birinchi topilgani ishlatiladi.



8-rasm. Bir nechta ota klass bo'lganda Python MRO tartibi bo'yicha metodni qidiradi va birinchi topilganini ishlatadi.

```
mro_olmos_muammosi.py

class A:
    def salom(self):
        return "A dan salom"

class B(A):
    def salom(self):
        return "B dan salom"

class C(A):
    def salom(self):
        return "C dan salom"

class D(B, C):
    # ikki tomondan meros — "olmos muammosi"
    pass

d = D()
print(d.salom())
print("MRO tartibi:")
for klass in D.__mro__:
    print("    ->", klass.__name__)

Natija (terminalda ishga tushirilgan)

B dan salom
MRO tartibi:
-> D
-> B
-> C
-> A
-> object
```

Qoidani eslab qolish oson: Python chapdan o'ngga (`class D(B, C)`) da avval B, keyin C) va pastdan yuqoriga qidiradi. Shuning uchun `D().salom()` "B dan salom" ni qaytardi.

7. Polimorfizm — ko'p ko'rinishlilik

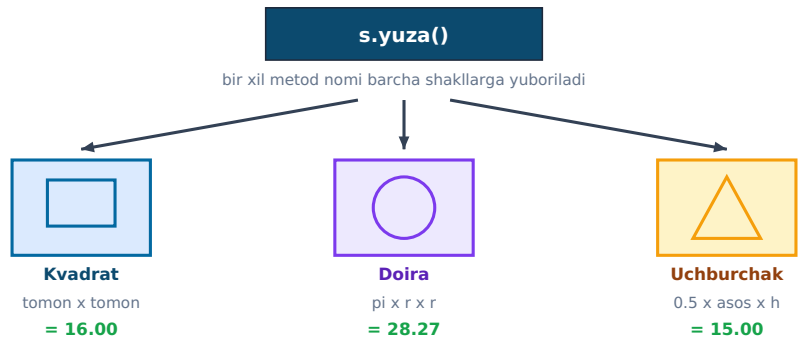
NIMA UCHUN KERAK?

Polimorfizm (yunoncha: *poly* — ko'p, *morphe* — shakl) — bir xil metod nomi turli obyektlarda o'zicha ishlashi.

Nima foyda? Chaqiruvchi kod **obyekt turini bilishi shart emas**. Sikl ichida shunchaki `s.yuza()` deb yozamiz — kvadratmi, doirami, uchburchakmi — har biri o'z formulasini o'zi qo'llaydi. Yangi shakl qo'shilsa ham, chaqiruvchi kodga **tegish shart emas**.

Bu — professional kodning asosiy belgisi: `if type(s) == Kvadrat: ... elif type(s) == Doira: ...` kabi uzun shartlar butunlay yo'qoladi.

POLIMORFIZM — bitta buyruq, turli natija



9-rasm. Bir xil metod nomi (yuza) har bir shaklda o'z formulasini bo'yicha ishlaydi — chaqiruvchi kod farqni bilishi shart emas.

polimorfizm.py

```
class Shakl:
    def yuza(self):
        raise NotImplementedError

class Kvadrat(Shakl):
    def __init__(self, tomon):
        self.tomon = tomon

    def yuza(self):
        return self.tomon ** 2

class Doira(Shakl):
    def __init__(self, radius):
        self.radius = radius

    def yuza(self):
        return 3.14159 * self.radius ** 2

class Uchburchak(Shakl):
    def __init__(self, asos, balandlik):
        self.asos = asos
        self.balandlik = balandlik

    def yuza(self):
        return 0.5 * self.asos * self.balandlik

# Bitta sikl — turli xil obyektlar, bir xil metod nomi
shakllar = [Kvadrat(4), Doira(3), Uchburchak(6, 5)]

for s in shakllar:
    print(f"{type(s).__name__:12} yuzasi = {s.yuza():.2f}")
```

Natija (terminalda ishga tushirilgan)

```
Kvadrat      yuzasi = 16.00
Doira        yuzasi = 28.27
Uchburchak   yuzasi = 15.00
```

7.1. Duck typing — Python'ning o'ziga xos uslubi

"O'RDAK NAZARIYASI"

Mashhur ibora: "Agar u o'rdakdek yursa va o'rdakdek qichqirsa — demak u o'rdak".

Java yoki C++ da polimorfizm uchun umumiy ota klass **majburiy**. Python'da esa shart emas — u faqat "**kerakli metod bormi?**" deb qaraydi. Klasslar bir-biri bilan umuman bog'liq bo'lmasa ham, bir xil nomli metodga ega bo'lsa — kod ishlayveradi.

duck_typing.py

```
class Odam:
    def gapir(self):
        return "Assalomu alaykum"

class Robot:
    def gapir(self):
        return "BIP-BOP-BIP"

class Mushuk:
    def gapir(self):
        return "Miyov"

# Hech qanday umumiy ota klass yo'q!
# Python faqat "gapir metodi bormi?" deb qaraydi
for obyekt in [Odam(), Robot(), Mushuk()]:
    print(f"{type(obyekt).__name__:8} -> {obyekt.gapir()}")
```

Natija (terminalda ishga tushirilgan)

```
Odam      -> Assalomu alaykum
Robot     -> BIP-BOP-BIP
Mushuk    -> Miyov
```

8. Abstraksiya

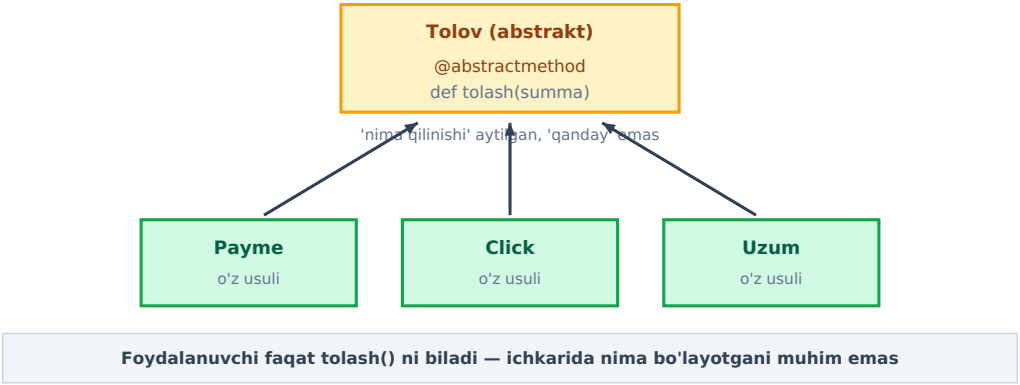
NIMA UCHUN KERAK?

Duck typing qulay, lekin xavfi bor: agar dasturchi `tolash()` metodini yozishni **unutib qolsa**, xato faqat dastur ishlagan paytda, foydalanuvchi oldida chiqadi.

Abstrakt klass — bu *shartnoma*: "mendan meros olgan har bir klass bu metodlarni albatta yozishi shart". Yozilmasa, obyekt yaratishning o'zi mumkin bo'lmaydi va xato darrov, kod yozish bosqichida ma'lum bo'ladi.

Bu `abc` (Abstract Base Class) moduli va `@abstractmethod` dekoratori orqali amalga oshiriladi.

ABSTRAKSIYA — murakkablikni yashirish



10-rasm. Abstrakt klass 'shartnoma' belgilaydi: har bir voris `tolash()` metodini albatta yozishi shart.

abstraksiya.py

```
from abc import ABC, abstractmethod

class Tolov(ABC):
    @abstractmethod
    def tolash(self, summa):
        pass

    def chek(self, summa):
        return f"Chek: {summa} so'm to'landi"

class Payme(Tolov):
    def tolash(self, summa):
        return f"Payme orqali {summa} so'm to'landi"

class Click(Tolov):
    def tolash(self, summa):
        return f"Click orqali {summa} so'm to'landi"
```

```
p = Payme()
print(p.tolash(50000))
print(p.chek(50000))

c = Click()
print(c.tolash(75000))

# Abstrakt klassdan obyekt yaratib bo'lmaydi
try:
    t = Tolov()
except TypeError as e:
    print("XATO:", e)
```

Natija (terminalda ishga tushirilgan)

```
Payme orqali 50000 so'm to'landi
Chek: 50000 so'm to'landi
Click orqali 75000 so'm to'landi
XATO: Can't instantiate abstract class Tolov without an implementation for abstract method 'tolash'
```

Natijaning oxirgi qatori muhim: `Tolov()` deb obyekt yaratmoqchi bo'lganda Python darhol `TypeError` berdi. Bu — himoya mexanizmi ishlayotganining isboti.

9. Qo'shimcha muhim imkoniyatlar

9.1. Magic (dunder) metodlar

NIMA UCHUN KERAK?

Ikki tomonida pastki chiziq bo'lgan metodlar (`__init__`, `__str__`) **magic** yoki **dunder** (double underscore) metodlar deyiladi. Ularni biz to'g'ridan-to'g'ri chaqirmaymiz — Python ma'lum vaziyatlarda **o'zi chaqiradi**.

Ular orqali o'z klassimizni Python'ning o'rnatilgan operatorlariga o'rgatish mumkin: `+` qanday ishlashi, `print()` nima chiqarishi, `==` qanday solishtirishi.

magic_metodlar.py

```
class Vektor:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vektor({self.x}, {self.y})"

    def __repr__(self):
        return f"Vektor(x={self.x}, y={self.y})"

    def __add__(self, boshqa):
        return Vektor(self.x + boshqa.x, self.y + boshqa.y)

    def __eq__(self, boshqa):
        return self.x == boshqa.x and self.y == boshqa.y

    def __len__(self):
        return int((self.x ** 2 + self.y ** 2) ** 0.5)
```

```
v1 = Vektor(3, 4)
v2 = Vektor(1, 2)
```

```
print(v1)           # __str__ ishlaydi
print(repr(v1))     # __repr__ ishlaydi
print(v1 + v2)      # __add__ ishlaydi
print(v1 == Vektor(3, 4)) # __eq__ ishlaydi
print(len(v1))      # __len__ ishlaydi
```

Natija (terminalda ishga tushirilgan)

```
Vektor(3, 4)
Vektor(x=3, y=4)
Vektor(4, 6)
True
5
```

ENG KO'P ISHLATILADIGANLARI

- `__init__` — obyekt yaratilganda ishlaydi (konstruktor)
- `__str__` — `print()` uchun chiroyli ko'rinish (foydalanuvchi uchun)
- `__repr__` — dasturchi uchun texnik ko'rinish (debug paytida)
- `__add__`, `__sub__` — `+` va `-` operatorlari
- `__eq__`, `__lt__` — `==` va `<` solishtirish
- `__len__` — `len()` funksiyasi

9.2. @staticmethod va @classmethod

UCH XIL METOD FARQI

- Oddiy metod** — birinchi parametri `self`. Obyektning ma'lumotlari bilan ishlaydi.
- @staticmethod** — na `self`, na `cls` oladi. Klass ichida joylashgan, lekin obyektga bog'liq bo'lmagan yordamchi funksiya (masalan, matematik hisob).
- @classmethod** — birinchi parametri `cls` (klassning o'zi). Ko'pincha obyektни boshqacha yo'l bilan yaratish uchun ishlatiladi ("muqobil konstruktor").

metod_turlari.py

```
class Matematika:
    PI = 3.14159

    def __init__(self, qiymat):
        self.qiymat = qiymat
```

```
def ikki_barobar(self):          # oddiy metod — self kerak
    return self.qiyamat * 2

@staticmethod
def qoshish(a, b):               # obyektga bog'liq emas
    return a + b

@classmethod
def noldan(cls):                 # klassning o'zi bilan ishlaydi
    return cls(0)

m = Matematika(10)
print("Oddiy metod:", m.ikki_barobar())
print("Static metod:", Matematika.qoshish(3, 7))
print("Class metod:", Matematika.noldan().qiyamat)
print("Klass atributi:", Matematika.PI)
```

Natija (terminalda ishga tushirilgan)

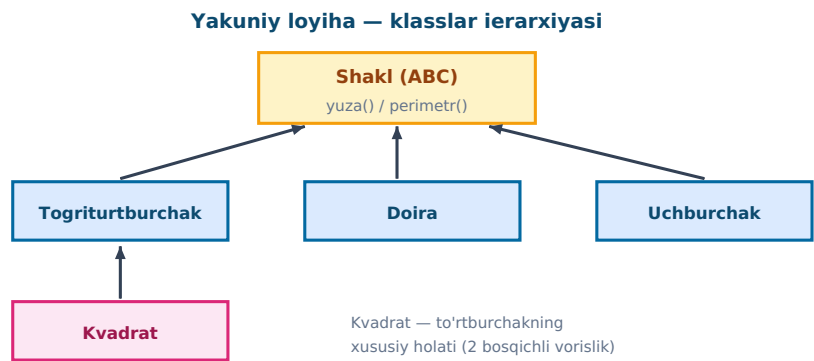
```
Oddiy metod: 20
Static metod: 10
Class metod: 0
Klass atributi: 3.14159
```

10. Yakuniy loyiha — Geometrik shakllar tizimi

LOYIHADA NIMALAR QO'LLANILADI?

Bu loyiha kursda o'rganilgan **barcha tushunchalarni bir joyda** birlashtiradi:

- **Abstraksiya** — Shakl(ABC) abstrakt klassi shartnoma belgilaydi
- **Vorislik** — uchta shakl Shakl dan meros oladi
- **Ko'p bosqichli vorislik** — Kvadrat → Togriturtburchak → Shakl
- **super()** — har bir voris ota konstruktorini chaqiradi
- **Polimorfizm** — bitta siklda barcha shakllarning yuza() metodi chaqiriladi
- **Magic metod** — __str__ chiroyli jadval ko'rinishini beradi



11-rasm. Yakuniy loyihaning klasslar tuzilishi: abstrakt Shakl klassidan uch xil shakl, Kvadrat esa To'rtburchakdan meros oladi.

```
geometrik_shakllar.py

from abc import ABC, abstractmethod
import math

class Shakl(ABC):
    """Barcha geometrik shakllar uchun asosiy abstrakt klass"""

    def __init__(self, nom):
        self.nom = nom

    @abstractmethod
    def yuza(self):
        pass

    @abstractmethod
    def perimetr(self):
        pass

    def __str__(self):
        return f"{self.nom:12} | yuza: {self.yuza():8.2f} | perimetr: {self.perimetr():8.2f}"

class Togriturtburchak(Shakl):
    def __init__(self, eni, boyi):
        super().__init__("To'rtburchak")
        self.eni = eni
        self.boyi = boyi

    def yuza(self):
        return self.eni * self.boyi

    def perimetr(self):
        return 2 * (self.eni + self.boyi)

class Kvadrat(Togriturtburchak):
    """Kvadrat — to'rtburchakning xususiy holati (vorislik zanjiri)"""

    def __init__(self, tomon):
        super().__init__(tomon, tomon)
        self.nom = "Kvadrat"

class Doira(Shakl):
    def __init__(self, radius):
        super().__init__("Doira")
        self.radius = radius

    def yuza(self):
        return math.pi * self.radius ** 2

    def perimetr(self):
        return 2 * math.pi * self.radius
```

```
class Uchburchak(Shakl):
    def __init__(self, a, b, c):
        super().__init__("Uchburchak")
        self.a, self.b, self.c = a, b, c

    def perimetr(self):
        return self.a + self.b + self.c

    def yuza(self):
        p = self.perimetr() / 2
        return math.sqrt(p * (p - self.a) * (p - self.b) * (p - self.c))

shakllar = [
    Togriturtburchak(5, 8),
    Kvadrat(6),
    Doira(4),
    Uchburchak(3, 4, 5),
]

print("=" * 55)
for s in shakllar:
    print(s)
print("=" * 55)

umumiy = sum(s.yuza() for s in shakllar)
print(f"Umumiy yuza: {umumiy:.2f}")

eng_katta = max(shakllar, key=lambda s: s.yuza())
print(f"Eng katta shakl: {eng_katta.nom}")

print(f"Kvadrat -> To'rtburchak vorisimi? {isinstance(Kvadrat(2), Togriturtburchak)}")
```

Natija (terminalda ishga tushirilgan)

```
=====
To'rtburchak | yuza:    40.00 | perimetr:    26.00
Kvadrat      | yuza:    36.00 | perimetr:    24.00
Doira        | yuza:    50.27 | perimetr:    25.13
Uchburchak   | yuza:     6.00 | perimetr:    12.00
=====
Umumiy yuza: 132.27
Eng katta shakl: Doira
Kvadrat -> To'rtburchak vorisimi? True
```

Diqqat qiling: Kvadrat klassida `yuza()` va `perimetr()` umuman yozilmagan! U ularni `Togriturtburchak` dan meros oldi va `super().__init__(tomon, tomon)` orqali eni va bo'yini teng qilib yubordi. Bu — vorislikning nozik va chiroyli qo'llanilishi.

11. Ko'p uchraydigan xatolar

TypeError: __init__() takes 2 positional arguments but 3 were given

Sabab: metodda `self` yozilmagan. Har bir oddiy metodning birinchi parametri `self` bo'lishi shart.

AttributeError: object has no attribute 'ism'

Sabab: voris klassda `__init__` yozilgan, lekin `super().__init__()` chaqirilmagan — shuning uchun ota klass atributlari o'rnatilmagan.

TypeError: Can't instantiate abstract class

Sabab: abstrakt klassdan to'g'ridan-to'g'ri obyekt yaratishga urinilgan, yoki voris klass barcha `@abstractmethod` larni yozmagan.

Barcha obyektlarda bir xil ro'yxat paydo bo'lishi

Sabab: o'zgaruvchan qiymat (list, dict) klass atributi sifatida yozilgan. Yechim: uni har doim `__init__` ichida `self.royxat = []` shaklida yarating.

O'qituvchi uchun maslahat: vorislikni tushuntirishda albatta **avval takrorlanuvchi kodni** ko'rsating — uchta klassda bir xil metodlarni yozib chiqing. O'quvchi "bu juda ko'p takror-ku" deb his qilgandan keyingina vorislikni kiriting. Muammoni ko'rmagan o'quvchi uchun yechim shunchaki sintaksis bo'lib qoladi.

12. Amaliy topshiriqlar

1-daraja (asos):

- Kitob klassini yarating: `nom`, `muallif`, `sahifa` atributlari va `malumot()` metodi bilan. 3 ta obyekt yaratib ekranga chiqaring.
- Kitob klassiga `__str__` magic metodini qo'shing va `print(Kitob)` ni sinang.

2-daraja (inkapsulyatsiya va vorislik):

- Xodim ota klassi va undan meros oladigan Menejer , Dasturchi klasslarini yozing. Har birida maosh_hisobla() metodi o'zicha ishlasin (masalan, menejerga bonus qo'shilsin).
- Maosh atributini __private qiling va unga @property orqali murojaat qiling. Manfiy maosh o'rnatilganda ValueError chiqsin.

3-daraja (to'liq loyiha):

- **Kutubxona tizimi** yarating: Mahsulot(ABC) abstrakt klassidan Kitob , Jurnal , DVD klasslari meros olsin. Har birida ijara_narxi() metodi o'z formulasi bo'yicha hisoblansin.
- Kutubxona klassi yarating: u mahsulotlar ro'yxatini saqlasin, qoshish() , royxat() va umumiy_narx() metodlariga ega bo'lsin.
- Polimorfizmdan foydalanib, bitta sikl orqali barcha mahsulotlarning ijara narxini chiqaring.
- **Qo'shimcha:** __len__ metodini qo'shib, len(kutubxona) mahsulotlar sonini qaytarsin.